Software Requirements Specification (SRS) Document

Cinema Web Application

The document was made by Samir Cibis, Daniel Helmrosin and Aleksander Kartuzov; K-TA-24B

1. Introduction

1.1 Purpose

The purpose of this document is to specify the requirements for the development of a cinema web application that allows users to view movie sessions, browse showtimes, and purchase tickets online.

1.2 Scope

This document outlines the functional and non-functional requirements for the Cinema Web Application. The system will provide features for users to search for movies, view available sessions, select seats, and complete ticket purchases.

2. General Description

2.1 Product Perspective

The Cinema Web Application will operate as a standalone system, interacting with internal/external databases for movie details, session schedules, and payment processing systems. It will provide a user-friendly interface for cinema customers and an administrative interface for cinema staff to manage sessions and ticketing.

2.2 Product Features

- **User Management:** User registration and login, user profile management.
- **Movie Listings:** Display of movies currently showing and upcoming releases.
- **Session Management:** The main page with info of the movies and will a big selection
- **Seat Selection:** Seat selection and ticket purchase, currently showing which seat has been occupied and which seats are available.
- **Ticket Purchase:** Section with a form, where you write your card information and choose a receiver.
- **Email Receiver:** After the previous section, this section will say where did your ticket was received and show an email or the QR-code ticket will be also generated, if the ticket wasn't received.

2.3 User Classes and Characteristics

- **Customers:** Individuals who browse movies, check session times, and purchase tickets.
- **Cinema Staff (Admin):** Users who manage movie listings, session schedules, and seat availability.
- The system consists of a Vue.js frontend and a NestJS backend connected to a MySQL relational database. The backend uses MikroORM for database access. The application can be deployed on a standard web server and accessed through a modern web browser.

2.4 Operating Environment

The web application will run on almost all modern web browsers that are compatible and works on newer versions of Chrome (version 109 or newer), Firefox (ver. 115 ESR or newer) and Safari (ver. 14 or newer). All browsers minimum requirements are Windows 7 / 8.1 or newer and macOS Sierra or newer.

All mobile phones require iOS 15 for a minimum requirement and Android 12 or newer software and hardware.

2.5 Design and Implementation Constraints

The application will use a relational database for managing movie data and user bookings.

The application will use a MySQL sphere and PrimeVue for managing database and design for developers.

3. Functional Requirements

3.1 Movie Listings

The system will display current and upcoming movies, including the following details:

- Movie title, poster, and description
- Rating and duration
- Available languages and subtitles
- Session times for each cinema location
- Creators of the movie

3.2 Session Management

The system will display available sessions for each movie, including the cinema hall, date, start time and language. Administrators can create, edit, and remove sessions, if they want or they need to do.

3.3 Seat Selection

The system will display available seats for each movie and there you can also choose your own place and then it will show you the cost of tickets and seats too. It depends on the seats (from the first row to last). Some seats can be booked already, so on this page it will not be available.

3.4 Ticket Purchase and E-mail receiver

After you choose your seat and you know what movie you'll go to, then you are going to a ticket purchase page. The system will automatically show you a form page, where you could write your email to receive a ticket and choose a category (how you can pay, Apple Pay, Card payment, etc.) After the filling of the form the next page will show you on which email your ticket has been sent. The system will also generate a QR-code as a QR ticket for comfortability.

4. Non-Functional Requirements

4.1 Performance

- The website's code should be more optimized with fixing some bugs or errors, if occurred due to adding some new content or previous problems occurred earlier.

4.2 Security

- Main pages should load and respond within a few seconds under normal usage. Seat selection and session data should update without unnecessary delays.

4.3 Usability

- The cinema website pages interface should be clear and understandable without unnecessary features or design "artefacts" due to code bugs. The user can easily orientate in the pages and understand what these features stand for so the user can use the website without any questions or misunderstandings.

4.4 Reliability

- The website should have the data encryption for preventing data theft for thieves and data transparency for threat prevention like malwares or data extortion. Also the website should load all content quickly without any problems and errors. The website must have some defence protocols to prevent data leaks or DDoS spam attacks as example. And very important is to have data backups if the data is unrecoverably lost.

4.5 Scalability

- With the increase in the number of users on the website it should work stably, all load quickly and requests and data processing efficiently. Totally, the website shouldn't lie down due to the large amount of users on the website.

4.6 Legal and Compliance

- The application must comply with GDPR for user data protection.
- The application should have the Privacy Policy to prevent any troubles with the privacy about website usage.
- The application should have cookie files for comfortable user's data saving and data preferences.

5. System Models

5.1 Use Case Diagram
A use case diagram will illustrate the interactions between customers, admin staff, and the system, such as browsing movies, selecting seats, and purchasing tickets.

5.2 Sequence Diagrams
Sequence diagrams will show the flow of actions for purchasing a ticket and managing session data.

5.3 Data Flow Diagrams
Data flow diagrams will depict the flow of information within the system, such as how movie session data is displayed to users and how bookings are processed.

5.4 Software System Architecture Diagram (SSAD):
Our cinema project uses a three tier client-server architecture (also it comes with Docker Engine as main site heart) The frontend shows the website, the backend handles the application system logic, and the database logic stores the site data. Also Docker Engine used to be an Engine for the whole project.

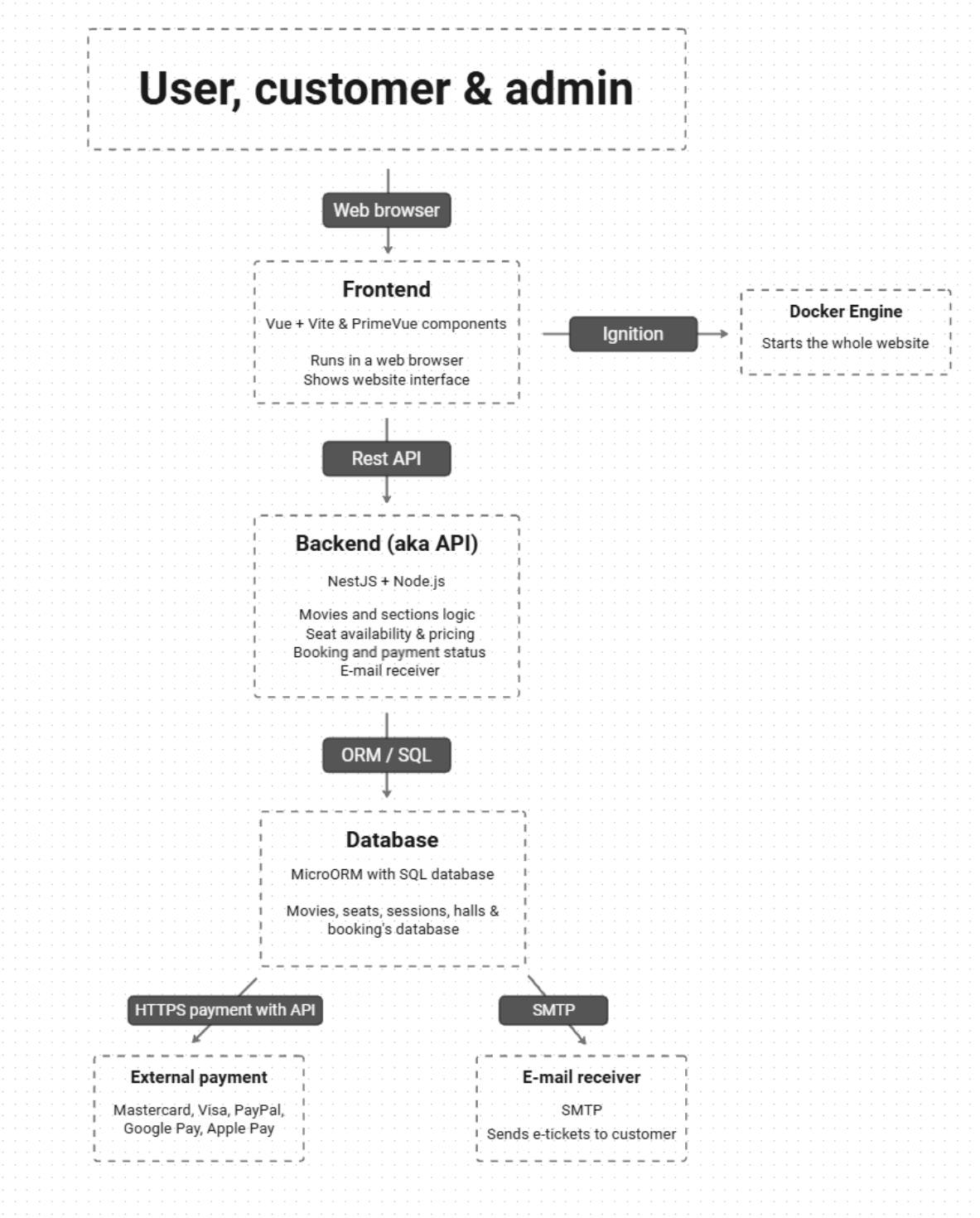
This architecture is suitable because each part has a clear responsibility. For example Vue.js and PrimeVue handle the user interface to use and show the design, NestJS processes movies, sessions, seats, bookings and ticket requests with continued payment. MicroORM connects the backend to MySQL and its own database structure. HTTPS/REST is used between the frontend and backend, while the payment and e-mail receivers use their own pages separately. All parts can be developed, tested and changed separately, so no one won’t collide with the incompatibilities and doesn’t require connection between each other.

The figure has been introduced down below:

Project Structure:

```
TA-24B-2/
│
├── frontend/                  # Vue.js + Vite + PrimeVue
│   ├── src/
│   │   ├── assets/             # Posters and banners
│   │   ├── services/
│   │   │   └── api.js          # Requests to the backend
│   │   ├── App.vue             # Main page and movie listings
│   │   ├── seatpicker.vue      # Session and seat selection
│   │   ├── Checkout.vue        # Checkout page
│   │   ├── router.js           # Website routes
│   │   └── main.js             # Starts the Vue application
│   └── package.json
│
├── backend/                   # NestJS + Node.js
│   ├── src/
│   │   ├── database/
│   │   │   └── mikro-orm.config.ts # Database connection settings
│   │   │
│   │   ├── movies/
│   │   │   ├── movies.module.ts      # Movies module
│   │   │   ├── movies.controller.ts # Handles API requests
│   │   │   ├── movies.service.ts    # Movie-related logic
│   │   │   ├── movie.entity.ts      # Movie data model
│   │   │   ├── movie-rating.entity.ts
│   │   │   ├── creator.entity.ts
│   │   │   ├── movie-creator.entity.ts
│   │   │   ├── cinema-building.entity.ts
│   │   │   └── screening.entity.ts
│   │   │
│   │   ├── app.module.ts          # Main application module
│   │   └── main.ts              # Starts the backend
│   │
│   ├── uploads/                # Uploaded movie posters
│   └── package.json
│
├── docker-compose.yml            # Runs MySQL using Docker
├── start.bat                  # Starts the project
├── README.md                     # Project description
├── guide.md                   # Setup and usage instructions
├── update-log.md                 # Change history
└── update.txt
```

How the components communicate:



6. External Interface Requirements

6.1 User Interfaces

- A user-friendly interface for customers to browse movies, view sessions, and buy tickets.
- An admin interface for cinema staff to manage movies and sessions.

6.2 Hardware Interfaces

- On PC, the website opens in the standard full-size version if the browser is full-sized. The user can navigate using a mouse and keyboard. The site can have the full width of the screen depending on the website structure and interface elements are large. On mobile, the website structure is compressing into the mobile screen being more compact - elements are vertical. The user can navigate using own finger (or stylus) to scroll the website up or down on the sensor screen. Same on the tablet, but due to bigger screen buttons are usually larger and the website might be wider than on the mobile. On the console the interface will be likely on PC, but using the controller to navigate, instead of the mouse and keyboard.

6.3 Software Interfaces

- Passwords must not be stored as plain text. Authentication must restrict access to protected functions ONLY FOR ADMINS, and user input must be validated on the backend. But the client should not register or login to the cinema page, because they only need to book a ticket, they will anyway receive it by email.

6.4 Communication Interfaces

- The interface should be simple and understandable. Users should be able to find a movie, choose a session, select seats, and complete a booking without unnecessary steps.

7. Testing and Validation

7.1 Test Plan

The backend and database structure should support adding more movies, sessions, users, and bookings without requiring major changes to the application.

Scope of testing — which parts of the system this plan covers, and at which test level: The testing covers the frontend, backend API, database interaction, authentication, movie listings, session management, seat selection, and ticket purchase. Functional testing will be performed at system and integration levels.

Features to be tested — Movie listing, user registration and login, session display, seat availability, seat selection, ticket booking, input validation, and admin movie/session management.

● Only necessary user data should be collected and users should be able to manage their account data.

Test approach — Testing will mainly be manual, supported by API testing where appropriate. Equivalence partitioning and boundary value analysis will be used for forms and input validation. State-based checks will be used for seat availability and booking status.

Entry criteria — The application must start successfully, the database must be available, test accounts must exist, required movies and sessions must be loaded, and the main frontend and backend functions must be accessible.

Exit criteria — All planned test cases must be executed. All Critical and Major defects must be fixed or documented and no blocking defects should remain in the main user flow.

Test environment and test data — Testing will use the local development environment with the Vue/Vite frontend, NestJS backend, and MySQL database. Test data will include sample users, movies, sessions, halls, seats, and bookings.

Risks and mitigations — Risks include database connection errors, incorrect seat availability, invalid test data, and server/API errors. Mitigations include prepared test data, repeated booking tests, and checking backend logs.

Deliverables and schedule — Deliverables include test cases, defect records, and a final test report. Testing will be completed before the final project demonstration and submission.

7.2 Test Cases

TC-05 – E-mail Receiver and QR Ticket. The test checks requirement 3.5. Enter a valid e-mail address and complete the purchase. The system should confirm that the ticket was sent to the entered e-mail and generate a QR code for the ticket. Status: Didn't develop yet.

TC-04 – Ticket Purchase. The test checks requirement 3.4. Select a movie, session and seat, then open the ticket purchase page. The selected movie, session, seats and total cost should be shown, and the payment option should be selectable. Status: Not executed.

TC-03 – Seat Selection. The test checks requirement 3.3. Open a session and go to seat selection. Available seats should be selectable, booked seats should not be selectable, and the ticket cost should change according to the selected seats. Status: Pass.

TC-02 – Session Management. The test checks requirement 3.2. Open a movie and view its sessions. The cinema hall, date, start time and language should be shown. An administrator should also be able to create, edit and remove a session. Status: Fail.

TC-01 – Movie Listings. The test checks requirement 3.1. Open the movie listings and select a movie. The title, poster, description, rating, duration, languages, subtitles, session times and creators should be shown. Status: Pass.

7.3 Validation Criteria

The system is validated when the five main functional requirements are working: movie listings, session management, seat selection, ticket purchase, and e-mail ticket delivery with a QR code. Administrative session management must also work as specified.

8. Maintenance and Support

8.1 Maintenance Procedures

Maintenance includes fixing software defects, updating dependencies when necessary, checking database integrity, and updating movie and session data. Changes should be tested before being deployed.

8.2 Support and Helpdesk

Support will be provided through the project team. Reported problems will be reproduced, documented, and fixed according to their severity. Users should provide the affected page, action, and error message when reporting a problem.

9. Appendices

9.1 Glossary

- SRS: Software Requirements Specification
- GDPR: General Data Protection Regulation BBQ
- CIoBT: Cinema Insider or Beta Tester.
- QR code: Quick Response code used as an electronic ticket identifier
- UI: User Interface
- API: Application Programming Interface
- E-MAIL: Electronic (Digital) Mail
- ORM: Object-Relational Mapping

9.2 Revision History

- Document revisions will be logged with dates and descriptions of changes.

- Also revisions will be logged into update-log.md in the project.

- It also can be done in other text documents, but those documents have to be in the main folder of the project and connected together with logs that also have a connection to Apollo Kino’s website.